Title: Deployment of a Java Web Application Using 2-Tier Architecture on AWS (Tomcat & RDS)

Prepared by: Sakshi Jain

---

1. Introduction

Cloud computing provides scalable, secure, and cost-efficient infrastructure for deploying enterprise applications. This project demonstrates hosting a Java-based student registration application using a **2-Tier AWS Architecture** consisting of:

- **Frontend & Application Logic** running on Apache **Tomcat EC2**
- **Backend Database** hosted in **RDS MySQL**
- **Secure access** via **Bastion/Jump Server**
- **VPC Networking** for segmentation, isolation, and security

This project showcases real-world professional deployment architecture used in modern IT companies.

---

2. Necessity of the Project

- Enterprise apps require **secure remote deployment**
- Physical servers have **high cost and limited scalability**
- Cloud models like AWS offer **high availability and redundancy**
- Corporations demand **zero downtime**, **backup**, and **security compliance**
- Enables **modern web access** from anywhere

---

3. Motivation

The project is inspired by real-time DevOps, Cloud & Infra roles:

- Industry uses Tomcat for Java hosting
- MySQL database with managed services like RDS
- Infrastructure automation, migration, and monitoring
- Secure networking with VPC, Public & Private Subnets

Learning these cloud deployment practices strongly improves job readiness.

4. Objectives

- Deploy a Java Web Application on a scalable platform
- Create secure AWS networking using **VPC + Subnets + Route Tables**
- Store and retrieve data from **RDS MySQL**
- Enable **Jump Server SSH access** instead of exposing backend servers publicly
- Demonstrate **End-to-End CI/CD-ready architecture**
- Learn secure cloud database connectivity

---

5. Literature Survey

Technology	**Purpose**	**Source Reference**
AWS EC2	VM compute platform	AWS Docs
Apache Tomcat	Java Servlet Container	Apache Documentation
RDS MySQL	Managed relational DB	AWS Docs
VPC	Security & Resource Isolation	AWS Cloud Architecture Guides
IAM	Access and credentials management	AWS IAM Best Practices

The study shows that **2-tier web systems are most commonly used** in early cloud adoption due to simplicity and cost-efficiency.

---

6. System Structure

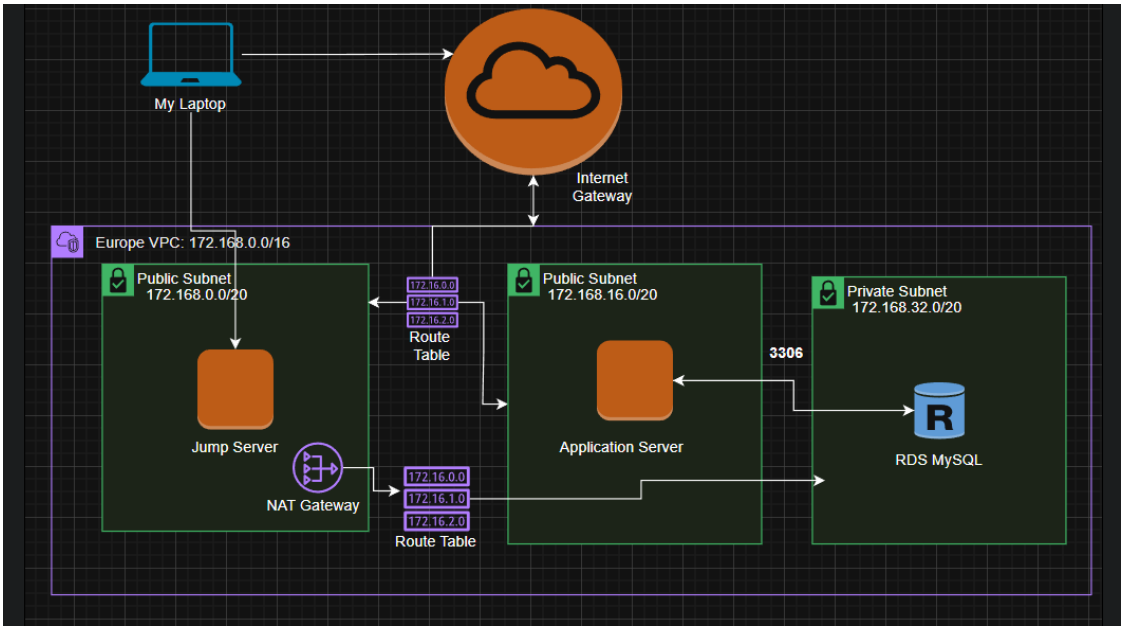
This architecture uses:

- **Public Subnet:** Jump Server + Application Server
- **Private Subnet:** RDS Database
- **Internet Gateway** for application access
- **NAT Gateway** for secure outbound traffic of private resources
- **Route Tables** for subnet-wise routing

---

7. System Design Framework

AWS Network Architecture Flow Diagram



Methodology

Step	Implementation
1	Create VPC, Subnets, Route Tables
2	Launch Jump Server in Public Subnet
3	Launch App Server & install Tomcat + Java
4	Create RDS MySQL in Private Subnet
5	Configure DB Subnet Group & SG rules
6	Deploy Java WAR (student.war)
7	Configure DB Connection in context.xml
8	Test end-to-end in browser
9	Secure SSH access using Key Pair + SG

roposed Learning Strategy

- Use **Hands-On practice** instead of theoretical learning
- Explore tools like:
    - AWS CLI

    - SSH & Key-based authentication
    - MySQL client commands
- Learn troubleshooting using Tomcat logs and RDS monitoring
- Repeat deployments for real-world expertise

---

8. Requirement Specification

Hardware/Cloud Requirements

Resource	Configuration
EC2 Instances	t2.micro (Free-Tier)
RDS MySQL	db.t2.micro
Storage	20–30GB gp2
Network	VPC with 3 subnets

Software Requirements

Component	Version
Apache Tomcat	9
MySQL Connector	JDBC Driver
Java JDK	openjdk 1.8+
Student WAR file	Provided

---

9. Results & Discussion

Test	Result
Tomcat runs and UI loads	✓ Successful
Database connectivity	✓ After correct SG + table
Data persistence	✓ Successfully stored and retrieved
Secure SSH	✓ Jump server enforced
Latency	Low inside same region

Main discussion point: misconfigured DB table name / SG blocked initial writes → resolved by correct MySQL creation.

---

10. Advantages

- Very secure architecture
- Scalable & cost-efficient
- RDS provides automated backups
- Jump server restricts SSH access
- Real-industry cloud deployment standards

---

11. Limitations / Disadvantages

- Manual deployment (no CI/CD yet)
- Single-AZ setup = not fault-tolerant by default
- No Load Balancer (scalability limited)

---

12. Applications

- University student portals
- Enterprise CRUD applications
- Corporate employee management tools
- Ticketing or registration systems
- Beginner DevOps/Cloud learning projects

---

13. Conclusion

The project successfully demonstrated secure deployment of a Java Web Application using AWS cloud infrastructure. A robust **2-Tier Client-Server Model** was implemented using Tomcat and RDS. The networking security and routing showcase professional real-time experience.

This project improves expertise in:

- Cloud computing
- Linux & SSH security
- Application server hosting
- Backend database integration
- Infrastructure design using AWS